

SONGO

Jeu Traditionnel Camerounais

Rapport complet : Version Locale et Version Distribuee

Analyse du jeu, Conception UML, Algorithmes,
Architecture logicielle, Code source, Documentation

Projet	Songo - Jeu de strategie camerounais
Technologies	HTML5, CSS3, JavaScript, AJAX, Node.js, Express
Versions	Locale (V1) + Distribuee (V2)

Table des matieres

Placeholder for table of contents	0
-----------------------------------	---

1. Fonctionnement du jeu

1.1 Presentation generale

Le Songo est un jeu de societe traditionnel camerounais appartenant a la famille des jeux de type mancala, aussi connus sous le nom d'Awale ou Oware en Afrique de l'Ouest. Ce jeu de strategie pure se joue a deux joueurs sur un plateau compose de 14 trous (7 par joueur) et de 70 graines. Chaque joueur dispose d'une rangee de 7 trous formeant son camp, et l'objectif est de capturer plus de graines que son adversaire au cours de la partie. Le Songo se distingue par des regles de capture precises et une profonde strategie remarquable pour un materiel aussi simple, ce qui en fait un jeu fascinant tant du point de vue culturel que mathematique.

Le jeu est traditionnellement joue sur un plateau en bois sculpte, souvent orne de motifs tribaux, avec des graines de type *Caesalpinia bonduc* (graines d'Awale) utilisees comme pions. La version numerique que nous presentons dans ce projet conserve l'esthetique africaine du jeu original tout en apportant les avantages de l'informatique : calcul automatique des scores, validation des coups, historique des parties, et possibilite de jouer a distance via une architecture client-serveur.

1.2 Description du plateau

Le plateau de Songo est constitue de deux rangees de 7 trous chacune, pour un total de 14 trous. Chaque trou peut contenir un nombre variable de graines. Le plateau est divise en deux camps : la rangee inferieure (trous 0 a 6) appartient au Joueur 1, et la rangee superieure (trous 7 a 13) appartient au Joueur 2. Une ligne de front separe les deux camps, marquant la frontiere entre les territoires de chaque joueur. Au debut de la partie, chaque trou contient exactement 5 graines, pour un total de 70 graines sur le plateau.

Element	Description	Valeur initiale
Trous par joueur	Chaque joueur possede une rangee de 7 trous	7
Nombre total de trous	Deux rangees de 7 trous	14
Graines par trou	Chaque trou contient 5 graines au debut	5
Graines totales	5 graines x 14 trous	70
Ligne de front	Separation entre les deux camps	-

Tableau 1 : Caracteristiques du plateau de Songo

1.3 Deroulement d'une partie

Une partie de Songo se deroule en tours alternes. Le Joueur 1 commence toujours la partie. A chaque tour, le joueur actif choisit un trou dans son propre camp, ramasse toutes les

graines qu'il contient, puis les distribue une par une dans les trous suivants, en suivant un parcours antihoraire autour du plateau. La distribution se fait en sautant le trou de depart (si le joueur fait un tour complet). Si la derniere graine distribuee tombe dans un trou du camp adverse contenant alors exactement 2 ou 3 graines (graine distribuee comprise), le joueur capture ces graines. La capture peut s'etendre en chaine aux trous precedents du camp adverse s'ils contiennent egalement 2 ou 3 graines. Cependant, un joueur ne peut pas capturer toutes les graines de l'adversaire en un seul coup (regle de securite).

La partie se termine dans l'une des conditions suivantes : un joueur ne peut plus jouer car tous les trous de son camp sont vides, ou il ne reste qu'une seule graine sur le plateau. Lorsqu'un joueur ne peut plus jouer, l'adversaire recupere toutes les graines restantes sur le plateau. Le joueur ayant capture le plus de graines remporte la partie. En cas d'egalite, la partie est declaree nulle.

1.4 Gestion des tours

La gestion des tours repose sur un mecanisme strict d'alternance. Apres que le Joueur 1 a effectue son coup, c'est au tour du Joueur 2, et inversement. Le systeme verifie automatiquement que le joueur actif choisit bien un trou de son propre camp et que ce trou n'est pas vide. De plus, une regle d'alimentation (feeding rule) impose qu'un joueur doit, si possible, jouer un coup qui laisse au moins une graine dans le camp adverse. Si le seul coup possible laisse l'adversaire sans graine mais qu'un autre coup est disponible, le coup invalide est refuse. Cette regle garantit que la partie reste dynamique et evite les situations de blocage premature.

1.5 Gestion des captures

La capture constitue le mecanisme central du Songo. Lorsqu'un joueur distribue ses graines et que la derniere graine tombe dans un trou du camp adverse, le systeme verifie si ce trou contient desormais 2 ou 3 graines. Si c'est le cas, le joueur capture ces graines. Ensuite, le systeme remonte les trous du camp adverse dans l'ordre inverse de la distribution : chaque trou contenant 2 ou 3 graines est egalement capture. Cette capture en chaine s'arrete des qu'un trou ne satisfait plus la condition (nombre de graines different de 2 et 3). Il existe egalement une regle de securite : un joueur ne peut pas capturer l'integralite des graines du camp adverse en un seul coup. Si la capture de tous les trous elegibles laisserait l'adversaire sans aucune graine, la capture s'arrete avant le dernier trou qui aurait cause cette situation.

1.6 Gestion des scores

Le score de chaque joueur correspond au nombre total de graines capturees au cours de la partie. Chaque capture ajoute immediatement les graines capturees au score du joueur actif. Le systeme met a jour l'affichage du score en temps reel apres chaque coup. En fin de partie, si un joueur ne peut plus jouer, l'adversaire recupere toutes les graines restantes

sur le plateau, qui s'ajoutent a son score. Le vainqueur est celui dont le score est le plus eleve. En pratique, le seuil de victoire est de 36 graines (plus de la moitie des 70 graines), bien que la partie puisse se terminer avant que toutes les graines ne soient capturees.

1.7 Conditions de victoire

La victoire est determinee selon les regles suivantes. Le joueur qui a capture le plus de graines a la fin de la partie est declare vainqueur. Si les deux joueurs ont capture le meme nombre de graines, la partie est declaree nulle. La partie prend fin dans les cas suivants : un joueur n'a plus de graines dans son camp et ne peut donc pas jouer ; il ne reste qu'une seule graine sur le plateau ; ou les joueurs conviennent d'un arre (en pratique, le systeme detecte automatiquement les fins de partie). Il est important de noter que la strategie optimale au Songo implique de planifier plusieurs coups a l'avance, en anticipant les captures et en evitant de donner trop de graines a l'adversaire.

2. Principe et histoire du Songo

2.1 Histoire du Songo

Le Songo, egalement connu sous les noms d'Oware, Awale, Awele ou Wari selon les regions, fait partie de la famille des jeux de mancala, parmi les plus anciens jeux de societe connus dans le monde. Les origines du mancala remontent a plusieurs millenaires, avec des traces archeologiques trouvees en Egypte ancienne, en Ethiopie et dans toute l'Afrique subsaharienne. Le Songo est la variante specifiquement pratiquee au Cameroun, ou il occupe une place centrale dans la culture ludique et sociale. Les plateaux traditionnels sont souvent sculptes dans le bois et transmis de generation en generation, constituant de veritables objets d'art.

Le jeu etait traditionnellement joue lors des gathering sociaux, dans les villages, apres les travaux champetres ou lors des ceremonies. Les parties pouvaient durer plusieurs heures et constituaient un veritable spectacle, les spectateurs commentant les coups et conseillant les joueurs. Les maitres du Songo etaient respects pour leur sagesse et leur capacite d'anticipation, qualites egalement recherchees chez les chefs de village et les diplomates. Le jeu servait ainsi de metaphore pour la gestion des ressources, la planification strategique et la resolution de conflits.

2.2 Importance culturelle

Le Songo depasse le simple cadre du divertissement pour revetir une dimension culturelle profonde au Cameroun. Il est considere comme un outil d'education qui enseigne la patience, la reflexion strategique, le calcul mental et la prise de decision. Les enfants apprennent le jeu des leurs parents et grands-parents, perpetuant ainsi une tradition orale et ludique seculaire. Le jeu est egalement associe a des proverbes et des enseignements

moraux : celui qui capture trop vite peut se retrouver sans ressources, symbolisant l'importance de la gestion a long terme plutôt que du profit immédiat.

Dans le contexte moderne, le Songo est de plus en plus reconnu comme un patrimoine culturel immatériel à préserver et à valoriser. Des tournois sont organisés au niveau national et international, et des recherches académiques ont montré que le jeu pouvait être utilisé comme outil pédagogique en mathématiques (arithmétique, théorie des jeux) et en informatique (algorithmique, intelligence artificielle). La numérisation du Songo, comme celle que nous présentons dans ce projet, contribue à cette préservation tout en rendant le jeu accessible à un public plus large.

2.3 Concepts stratégiques

La stratégie au Songo repose sur plusieurs principes fondamentaux que tout joueur doit maîtriser. Premièrement, le comptage des graines est essentiel : un bon joueur connaît à tout moment le nombre de graines dans chaque trou et peut anticiper les résultats de ses coups. Deuxièmement, la gestion des gros trous (trous contenant beaucoup de graines) est cruciale, car un gros trou permet de distribuer des graines sur une grande distance et de créer des opportunités de capture. Troisièmement, la règle d'alimentation impose de toujours laisser des graines à l'adversaire, ce qui oblige à planifier ses coups pour éviter de se retrouver dans une situation où l'on ne peut que rendre des graines.

Parmi les stratégies avancées, on trouve le 'piège' qui consiste à préparer un trou avec précisément 1 ou 2 graines de manière à forcer l'adversaire à y déposer des graines qui seront ensuite capturées. Le 'contre-piège' consiste à anticiper le piège adverse et à utiliser un coup qui neutralise la menace. La maîtrise de ces stratégies nécessite une grande pratique et une excellente capacité de calcul mental, ce qui fait du Songo un jeu d'une richesse stratégique comparable aux échecs ou au jeu de Go.

2.4 Choix de conception

Pour la réalisation de ce projet, plusieurs choix de conception ont été effectués afin de garantir la qualité, la maintenabilité et l'évolutivité du logiciel. Premièrement, nous avons adopté une architecture modulaire séparant clairement la logique métier (SongoBoard, SongoGame) de l'interface utilisateur (SongoUI) et de la persistance (SongoStorage). Cette séparation permet de modifier l'interface sans affecter les règles du jeu, et inversement.

Deuxièmement, le choix de HTML5/CSS3/JavaScript pour la version locale garantit une accessibilité maximale sans nécessité d'installation. Pour la version distribuée, l'utilisation d'AJAX via XMLHttpRequest respecte les contraintes technologiques imposées tout en assurant une communication efficace entre les clients et le serveur. Le serveur Node.js avec Express offre une solution légère et performante pour la gestion des sessions de jeu. Enfin, l'interface utilisateur s'inspire de l'esthétique africaine traditionnelle avec des tons terreux, des motifs tribaux et un plateau au rendu bois, tout en conservant une ergonomie moderne.

3. Algorithmes

3.1 Algorithme d'initialisation du plateau

Principe

L'initialisation du plateau consiste a creer un tableau de 14 elements, chacun initialise a la valeur 5 (5 graines par trou). L'algorithme reinitialise egalement les scores a zero, le joueur courant a 1, l'historique des coups a vide et le statut du jeu a 'playing'. Cette operation est executee au demarrage de chaque nouvelle partie et lors du redemarrage.

Pseudo-code

```
Fonction initPlateau():  
trous = tableau de 14 elements  
Pour i de 0 a 13:  
trous[i] = 5  
joueurCourant = 1  
scores = {1: 0, 2: 0}  
historique = []  
statut = 'playing'  
Retourner trous
```

Complexite

Complexite	Valeur	Justification
Temporelle	O(n) avec n=14	Parcours lineaire de 14 elements
Memoire	O(n)	Stockage du tableau de 14 elements

3.2 Algorithme de distribution des graines

Principe

L'algorithme de distribution est le coeur du mecanisme de jeu. Lorsqu'un joueur choisit un trou, l'algorithme ramasse toutes les graines de ce trou, puis les distribue une par une dans les trous suivants selon un ordre antihoraire predefini. Pour le Joueur 1, l'ordre est : 0, 1, 2, 3, 4, 5, 6, 13, 12, 11, 10, 9, 8, 7. Pour le Joueur 2, l'ordre est : 13, 12, 11, 10, 9, 8, 7, 0, 1, 2, 3, 4, 5, 6. Le trou de depart est saute si le joueur fait un tour complet du plateau.

Pseudo-code

```
Fonction distribuer(trous, indexTrou, joueur):  
graines = trous[indexTrou]  
trous[indexTrou] = 0  
ordre = getOrdreDistribution(joueur)  
posDepart = indexDe(indexTrou, ordre)  
courant = indexTrou
```

```

grainesRestantes = graines
Pour i de 1 a grainesRestantes:
    prochainePos = (posDepart + i) modulo 14
    courant = ordre[prochainePos]
    Si courant == indexTrou ET i < grainesRestantes:
        grainesRestantes = grainesRestantes + 1
    Continuer
    trous[courant] = trous[courant] + 1
Retourner courant // dernier trou

```

Complexite

Complexite	Valeur	Justification
Temporelle	$O(k)$ avec k = nombre de graines	Chaque graine est distribuee exactement une fois
Memoire	$O(1)$	Pas de structure auxiliaire supplementaire

3.3 Algorithme de capture des graines

Principe

L'algorithme de capture s'exécute après la distribution. Il vérifie si la dernière graine est tombée dans le camp adverse et si le trou contient 2 ou 3 graines. Si oui, il capture ces graines et remonte les trous précédents dans le camp adverse (capture en chaîne). La capture s'arrête lorsqu'un trou ne contient pas 2 ou 3 graines, ou lorsque la capture laisserait l'adversaire sans aucune graine (règle de sécurité).

Pseudo-code

```

Fonction capturer(trous, dernierTrou, joueur):
    adversaire = 3 - joueur
    trousAdversaire = getTrousJoueur(adversaire)
    captures = []
    Si dernierTrou n'appartient pas a adversaire:
        Retourner captures // pas de capture
    Si trous[dernierTrou] != 2 ET trous[dernierTrou] != 3:
        Retourner captures // condition non remplie
    Pour chaque trou en remontant depuis dernierTrou:
        Si trou n'appartient pas a adversaire: Arrêter
        Si trous[trou] == 2 OU trous[trou] == 3:
            // Verifier regle de securite
            testerCapture = trous.clone()
            Pour chaque c dans captures: testerCapture[c] = 0
            testerCapture[trou] = 0
            Si testerCapture.grainesJoueur(adversaire) > 0:
                captures.ajouter(trou)
            Sinon:
                Arrêter // ne pas affamer l'adversaire

```

Sinon:
Arreter
Retourner captures

Complexite

Complexite	Valeur	Justification
Temporelle	$O(m)$ avec $m = 7$ max	Au plus 7 trous dans le camp adverse
Memoire	$O(m)$	Liste des captures, au plus 7 elements

3.4 Algorithme de validation des coups

Principe

L'algorithme de validation verifie qu'un coup est legal avant de l'executer. Les criteres de validation sont : le jeu est en cours, c'est bien le tour du joueur, le trou selectionne appartient au joueur, le trou n'est pas vide, et le coup respecte la regle d'alimentation. Pour verifier cette derniere, l'algorithme simule le coup et verifie si l'adversaire aura au moins une graine apres le coup. Si ce n'est pas le cas, l'algorithme cherche s'il existe un autre coup qui permettrait de nourrir l'adversaire.

Pseudo-code

```
Fonction validerCoup(indexTrou, joueur):  
Si statut != 'playing': Retourner invalide  
Si joueur != joueurCourant: Retourner invalide  
Si trous[indexTrou] == 0: Retourner invalide  
Si trou n'appartient pas a joueur: Retourner invalide  
// Verifier regle d'alimentation  
resultat = simulerCoup(indexTrou, joueur)  
Si resultat.grainesAdversaireApres == 0:  
coupNourrissant = trouverCoupNourrissant(joueur)  
Si coupNourrissant != -1 ET coupNourrissant != indexTrou:  
Retourner invalide // doit nourrir l'adversaire  
Retourner valide
```

Complexite

Complexite	Valeur	Justification
Temporelle	$O(n \times k)$ au pire	Simulation de n trous avec k graines chacun
Memoire	$O(n)$	Clone du plateau pour la simulation

3.5 Algorithme de calcul des scores

Principe

Le calcul des scores est incremental. A chaque capture, les graines capturees sont ajoutees au score du joueur actif. En fin de partie, si un joueur ne peut plus jouer, toutes les graines restantes sur le plateau sont attribuees a son adversaire. L'algorithme parcourt simplement les trous du plateau et somme les graines restantes pour les ajouter au score de l'adversaire du joueur qui ne peut plus jouer.

Pseudo-code

```
Fonction calculerScores():
  Pour chaque trou i de 0 a 13:
    Si trous[i] > 0:
      scores[adversaire] += trous[i]
      trous[i] = 0
  Si scores[1] > scores[2]: vainqueur = 1
  Sinon Si scores[2] > scores[1]: vainqueur = 2
  Sinon: vainqueur = 'draw'
```

Complexite

Complexite	Valeur	Justification
Temporelle	$O(n)$ avec $n=14$	Parcours des 14 trous
Memoire	$O(1)$	Pas de structure auxiliaire

3.6 Algorithme de detection de fin de partie

Principe

L'algorithme de detection de fin de partie verifie deux conditions apres chaque coup : premierement, si le joueur dont c'est le tour n'a plus aucune graine dans son camp (tous ses trous sont vides), la partie est terminee et l'adversaire recupere les graines restantes ; deuxiemement, s'il ne reste qu'une seule graine sur le plateau, la partie est egalement terminee. Dans les deux cas, les graines restantes sont attribuees au joueur qui peut encore jouer, et le vainqueur est determine par comparaison des scores.

Pseudo-code

```
Fonction verifierFinPartie():
  grainesJoueurCourant = sommeGraines(joueurCourant)
  Si grainesJoueurCourant == 0:
    // L'adversaire recupere les graines restantes
    Pour i de 0 a 13:
      scores[adversaire] += trous[i]
      trous[i] = 0
    statut = 'finished'
    determinerVainqueur()
  Retourner vrai
  Si grainesTotales() <= 1:
    attribuerGrainesRestantes(joueurCourant)
```

```
statut = 'finished'
determinerVainqueur()
Retourner vrai
Retourner faux
```

Complexite

Complexite	Valeur	Justification
Temporelle	O(n) avec n=14	Parcours des trous pour compter les graines
Memoire	O(1)	Pas de structure auxiliaire

3.7 Algorithmes AJAX pour la version distribuee

Principe de communication AJAX

La version distribuee utilise AJAX (Asynchronous JavaScript and XML) via l'objet XMLHttpRequest pour toutes les communications entre le client et le serveur. Le principe repose sur un mecanisme de polling : le client interroge periodiquement le serveur pour obtenir l'etat du jeu, et envoie ses coups via des requetes HTTP POST asynchrones. Ce mecanisme permet de maintenir les deux clients synchronises sans necessiter de connexion WebSocket, conformement aux exigences du projet.

Algorithme de polling

```
Fonction pollerEtatJeu():
Toutes les 1000 ms:
xhr = new XMLHttpRequest()
xhr.open('GET', '/api/game-state?room=' + codeSalon)
xhr.onreadystatechange = function():
Si xhr.readyState == 4 ET xhr.status == 200:
etat = JSON.parse(xhr.responseText)
miseAJourPlateau(etat)
miseAJourScores(etat.scores)
Si etat.statut == 'finished':
afficherFinPartie(etat.vainqueur)
xhr.send()
```

Algorithme d'envoi de coup

```
Fonction envoyerCoup(indexTrou):
xhr = new XMLHttpRequest()
xhr.open('POST', '/api/make-move')
xhr.setRequestHeader('Content-Type', 'application/json')
xhr.onreadystatechange = function():
Si xhr.readyState == 4:
Si xhr.status == 200:
reponse = JSON.parse(xhr.responseText)
Si reponse.success:
```

```

miseAJourPlateau(reponse.etat)
Sinon:
afficherErreur(reponse.reason)
Sinon:
gererErreurReseau()
xhr.send(JSON.stringify({
  roomCode: codeSalon,
  playerNumber: numeroJoueur,
  holeIndex: indexTrou
}))

```

Complexite des communications

Operation	Complexite reseau	Frequence	Donnees transferees
Polling etat	O(1) par requete	Toutes les 1s	~200 octets JSON
Envoi coup	O(1) par requete	Par coup joue	~100 octets JSON
Verification connexion	O(1) par requete	Toutes les 5s	~50 octets JSON
Creation salon	O(1)	1 fois	~150 octets JSON

4. Organisation du code

4.1 Structure du projet

Le projet Songo est organise en deux repertoires principaux correspondant aux deux versions du jeu, ainsi qu'un repertoire de documentation et un repertoire de rapports. Cette structure modulaire facilite la maintenance, les tests et l'evolution independante de chaque version. Le repertoire version_locale contient l'ensemble des fichiers necessaires a la version jouee sur une seule machine, tandis que le repertoire version_distante contient les fichiers client et serveur pour la version en reseau.

```

/songo
|-- version_locale/
| |-- html/index.html // Page principale
| |-- css/style.css // Feuille de style
| |-- js/songo.js // Logique du jeu
| |-- assets/ // Images et ressources
|
|-- version_distante/
| |-- client/
| | |-- html/index.html // Interface client
| | |-- css/style.css // Style client
| | |-- js/songo-client.js // Logique client + AJAX
| | |-- assets/ // Ressources client

```

```
| |-- serveur/  
| | |-- server.js // Serveur Express  
| | |-- game-logic.js // Logique serveur  
| | |-- package.json // Dependances  
| |-- ajax/  
| | |-- ajax-manager.js // Module AJAX  
| |-- api/  
| | |-- api-routes.js // Routes API REST  
|  
|-- documentation/ // Diagrammes UML  
|-- rapports/ // Rapports et documents
```

4.2 Description des modules

Module SongoBoard

Le module SongoBoard est responsable de la gestion de l'etat du plateau de jeu. Il encapsule le tableau de 14 trous et fournit des methodes d'accès, de modification, de clonage et de serialisation. Ce module est le fondement sur lequel repose toute la logique du jeu. Il ne contient aucune logique de jeu propre, mais offre les primitives necessaires aux autres modules. Les methodes principales incluent `init()` qui initialise le plateau, `getPlayerHoles()` qui retourne les indices des trous d'un joueur, `getTotalSeeds()` qui calcule le nombre total de graines, et `clone()` qui cree une copie independante du plateau pour les simulations.

Module SongoGame

Le module SongoGame implemente l'ensemble des regles du Songo. Il utilise une instance de SongoBoard pour gerer le plateau et ajoute les couches de logique suivantes : validation des coups (`validateMove`), execution des coups avec distribution et capture (`makeMove`), calcul des captures avec la regle de securite (`calculateCaptures`), gestion des tours (`switchTurn`), detection de fin de partie (`checkGameOver`), et determination du vainqueur (`determineWinner`). Ce module est le coeur du jeu et est utilise tel quel dans la version locale, tandis que la version distribuee utilise un module equivalent cote serveur (`game-logic.js`).

Module SongoUI

Le module SongoUI gere toutes les interactions avec l'utilisateur via le DOM. Il cree et met a jour les elements visuels du plateau, gere les evenements de clic sur les trous, anime la distribution et la capture des graines, affiche les scores et les messages, et gere les modales (aide, parametres, fin de partie). Ce module est le seul a interagir directement avec le navigateur et depend des modules SongoGame, SongoHistory et SongoStorage pour les operations non visuelles.

Module SongoHistory

Le module SongoHistory gere l'affichage de l'historique des coups dans le panneau dedie. Chaque coup est enregistre avec son numero, le joueur, le trou selectionne, le nombre de graines distribuees, les captures effectuees et le nombre de graines capturees. L'historique permet aux joueurs de suivre le deroulement de la partie et d'analyser leurs choix strategiques.

Module SongoStorage

Le module SongoStorage gere la persistance de l'etat du jeu via le mecanisme localStorage du navigateur. Il permet de sauvegarder et de charger une partie en cours, y compris le plateau, les scores, le joueur courant, l'historique des coups et les parametres. La serialisation utilise le format JSON pour une compatibilite maximale.

4.3 Description des classes et fonctions

Classes de la version locale

Classe	Attributs principaux	Methodes principales
SongoBoard	holes[14], HOLES_PER_ROW, TOTAL_HOLES	init(), getPlayerHoles(), getTotalSeeds(), clone(), serialize()
SongoGame	board, currentPlayer, scores, moveHistory, gameStatus, winner	validateMove(), makeMove(), calculateCaptures(), checkGameOver(), serialize()
SongoUI	game, history, storage, playerNames, animating, settings	buildBoard(), renderBoard(), handleHoleClick(), animateMove(), saveGame()
SongoHistory	container	addMove(), clear(), rebuild()
SongoStorage	key	save(), load(), hasSave(), clearSave()

Tableau 2 : Classes de la version locale

Classes de la version distribuee

Classe	Role	Methodes principales
SongoClient	Logique client + rendu	connect(), createRoom(), sendMove(), pollState(), renderBoard()
AjaxManager	Communication HTTP	get(), post(), poll(), abort()
GameLogic (serveur)	Regles du jeu cote serveur	validateMove(), distributeSeeds(), captureSeeds(), checkGameOver()

Tableau 3 : Classes de la version distribuee

4.4 Interactions entre composants

Dans la version locale, les interactions suivent un schema clair : SongoUI intercepte les evenements utilisateur et appelle SongoGame pour la validation et l'execution des coups. SongoGame utilise SongoBoard pour manipuler le plateau et SongoHistory pour enregistrer les coups. SongoStorage est appele par SongoUI pour les operations de sauvegarde et de chargement. Ce schema garantit une separation stricte des responsabilites.

Dans la version distribuee, le schema est enrichi par la couche reseau. SongoClient remplace SongoUI pour les interactions avec le serveur. Lorsqu'un joueur clique sur un trou, SongoClient utilise AjaxManager pour envoyer le coup au serveur via une requete POST. Le serveur traite le coup via GameLogic, met a jour l'etat du jeu, et retourne le resultat. Le client adverse recoit la mise a jour via le mecanisme de polling (GET periodique). Les deux clients restent ainsi synchronises sans necessiter de connexion persistante.

4.5 Justification des choix techniques

- **HTML5/CSS3/JavaScript** : Technologies web standards, ne necessitant aucune installation, compatibles avec tous les navigateurs modernes, et permettant un deploiement immediat.
- **Architecture modulaire** : Separation claire entre logique metier, interface utilisateur et persistance, facilitant les tests unitaires et la maintenance.
- **AJAX (XMLHttpRequest)** : Respect de la contrainte technologique imposee, communication asynchrone sans rechargement de page, compatibilite universelle.
- **Node.js/Express** : Runtime JavaScript cote serveur leger et performant, ideal pour les applications temps reel, ecosysteme riche via npm.
- **Polling (1 seconde)** : Solution simple et fiable pour la synchronisation, adaptee au rythme d'un jeu de plateau tour par tour ou les delais d'une seconde sont acceptables.
- **localStorage** : Persistence native du navigateur, sans dependance externe, adaptee a la sauvegarde de l'etat du jeu entre les sessions.

5. Conception UML

Ce chapitre presente les diagrammes UML realises pour la conception du jeu Songo. Ces diagrammes couvrent les aspects statiques (cas d'utilisation, classes) et dynamiques (sequences, activites) du systeme, ainsi que les architectures des deux versions et le modele de donnees.

5.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les acteurs du systeme et les fonctionnalites qu'ils peuvent utiliser. Trois acteurs sont identifies : le Joueur Local (version 1), le Joueur

Distant (version 2) et le Serveur (version 2). Les cas d'utilisation couvrent les fonctionnalités communes aux deux versions (jouer un coup, valider, capturer, consulter le score, sauvegarder, charger, démarrer une partie, consulter l'historique) et les fonctionnalités spécifiques à la version distribuée (connexion, création de session, synchronisation, gestion des déconnexions).

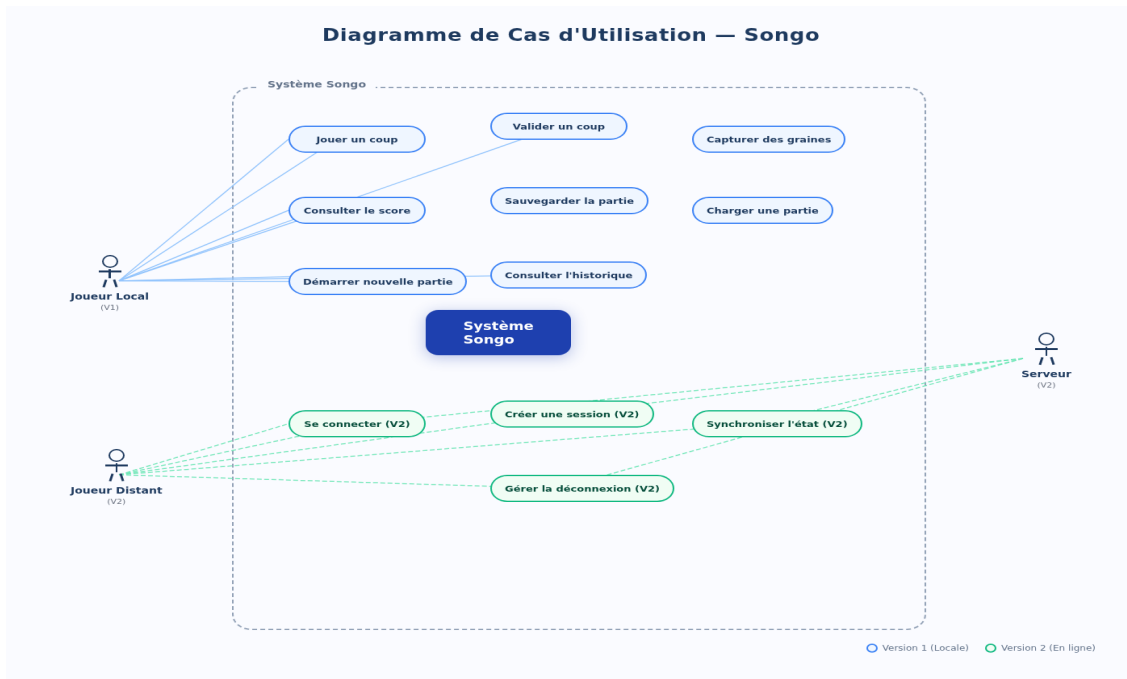


Figure 1 : Diagramme de cas d'utilisation du système Songo

5.2 Diagramme de classes

Le diagramme de classes présente la structure statique du système avec les 8 classes principales et leurs relations. Les classes SongoBoard, SongoGame, SongoUI, SongoHistory et SongoStorage forment le cœur de la version locale. Les classes SongoClient et AjaxManager constituent le client de la version distribuée, tandis que GameLogic représente la logique serveur. Les relations de composition et de dépendance montrent comment les modules interagissent.

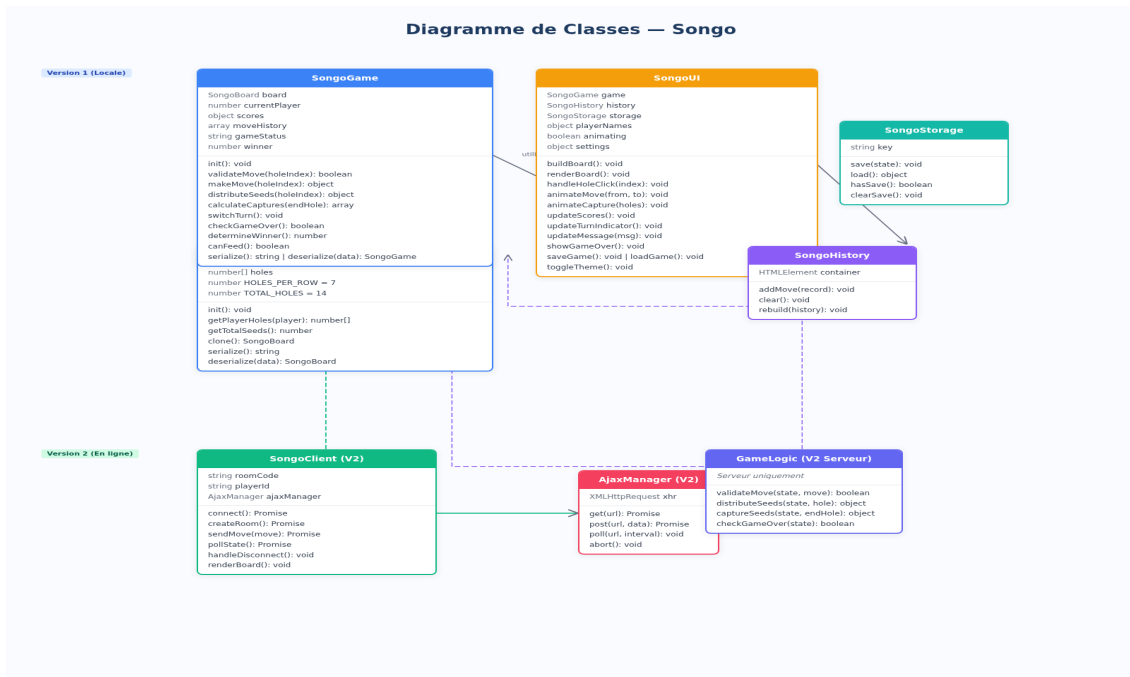


Figure 2 : Diagramme de classes UML du système Songo

5.3 Diagramme de sequence - Version locale

Le diagramme de sequence de la version locale montre le déroulement d'un coup complet depuis la sélection du trou par le joueur jusqu'à la mise à jour de l'interface. Les cinq phases sont : sélection du coup, distribution des graines, capture, mise à jour de l'interface, et vérification de fin de partie. Ce diagramme illustre la communication entre les modules SongoUI, SongoGame et SongoBoard.

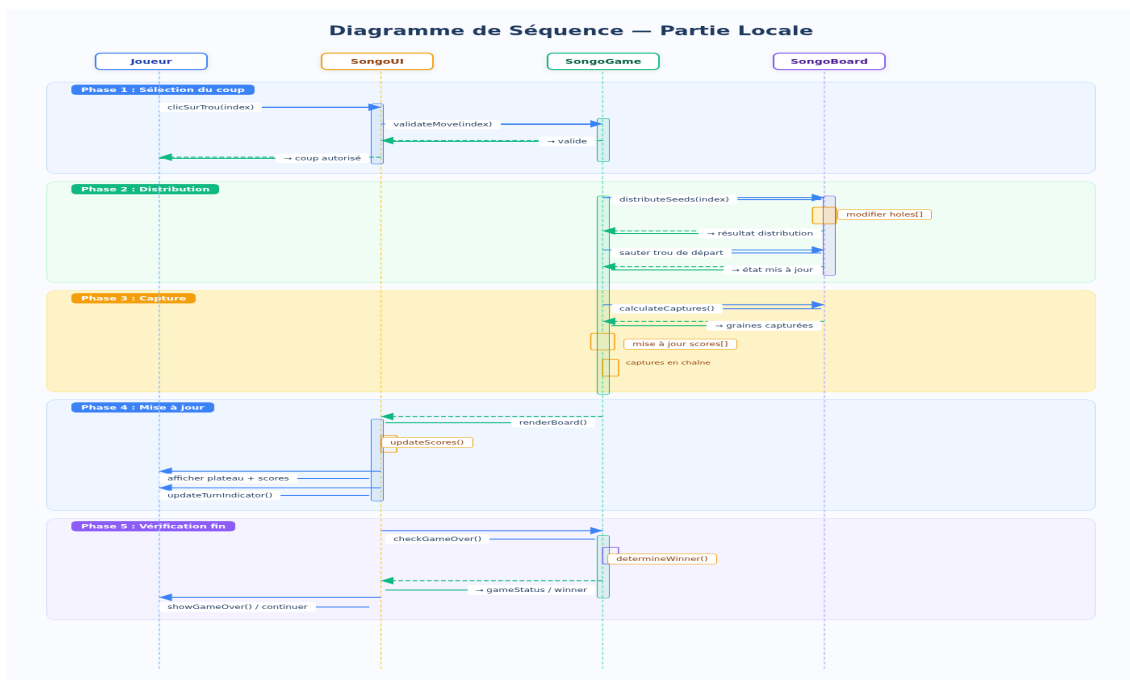


Figure 3 : Diagramme de sequence - Version locale

5.4 Diagramme de sequence - Version distribuee

Le diagramme de sequence de la version distribuee montre le flux de communication entre les clients et le serveur via AJAX. Les phases incluent : connexion et creation du salon, jointure du deuxieme joueur, envoi d'un coup via POST, synchronisation par polling GET, et detection de fin de partie. Ce diagramme met en evidence l'utilisation d'AJAX comme mecanisme de communication central entre les composants distribues.

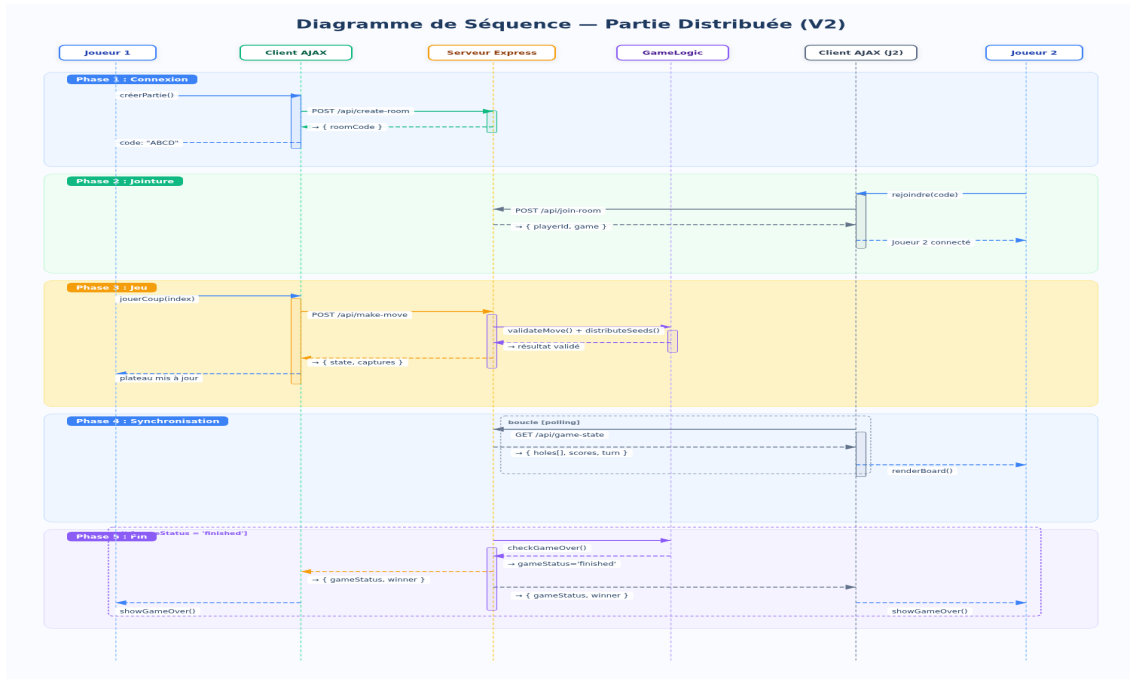


Figure 4 : Diagramme de sequence - Version distribuee

5.5 Diagramme d'activite

Le diagramme d'activite decrit le flux de controle lors d'un tour de jeu complet. Il montre les decisions prises a chaque etape : validation du coup, distribution des graines, verification des captures, application de la regle de securite, changement de tour ou detection de fin de partie. Les boucles et les branchements conditionnels illustrent la complexite du mecanisme de jeu.

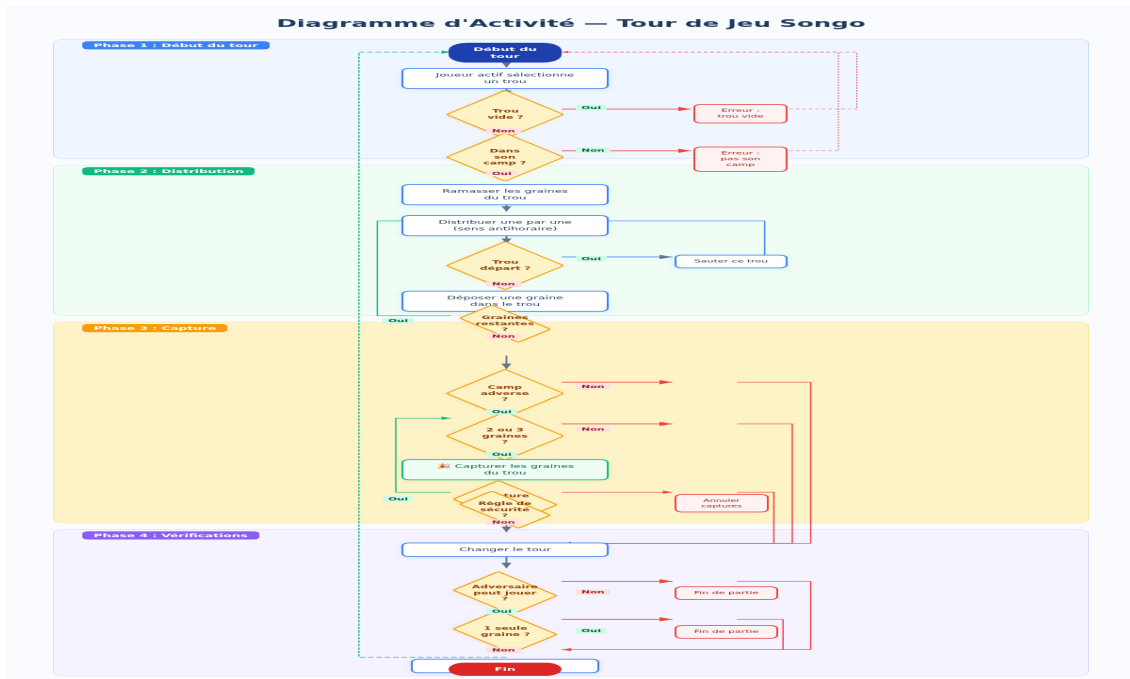


Figure 5 : Diagramme d'activité d'un tour de jeu

5.6 Architecture de la version locale

L'architecture de la version locale est organisée en trois couches : la couche de présentation (HTML5 + CSS3) responsable de l'affichage, la couche logique (SongoGame, SongoBoard, SongoHistory) implementant les règles du jeu, et la couche de stockage (SongoStorage + localStorage) assurant la persistance. Cette architecture en couches garantit une separation claire des responsabilites et facilite les evolutions futures.

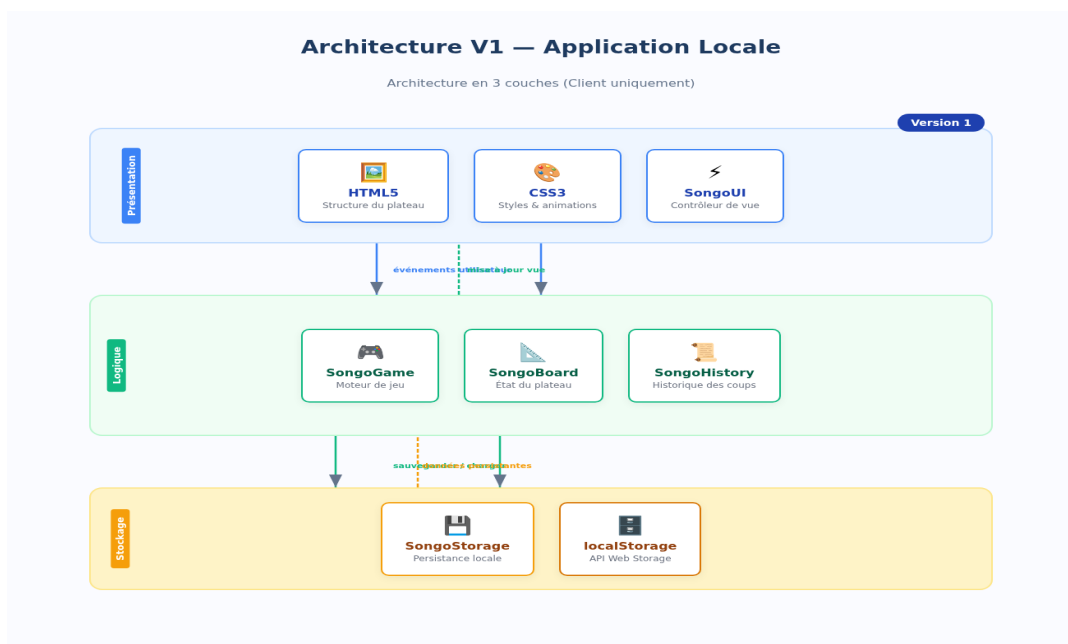


Figure 6 : Architecture en couches de la version locale

5.7 Architecture de la version distribuee

L'architecture de la version distribuee adopte un modele client-serveur. Les clients executent le code HTML/CSS/JavaScript dans le navigateur et communiquent avec le serveur Node.js/Express via des requetes AJAX (XMLHttpRequest). Le serveur gere les sessions de jeu, applique les regles via le module GameLogic, et stocke les etats en memoire. Les quatre couches (Client, Reseau, Serveur, Stockage) sont clairement separees, et chaque couche n'interagit qu'avec les couches adjacentes.

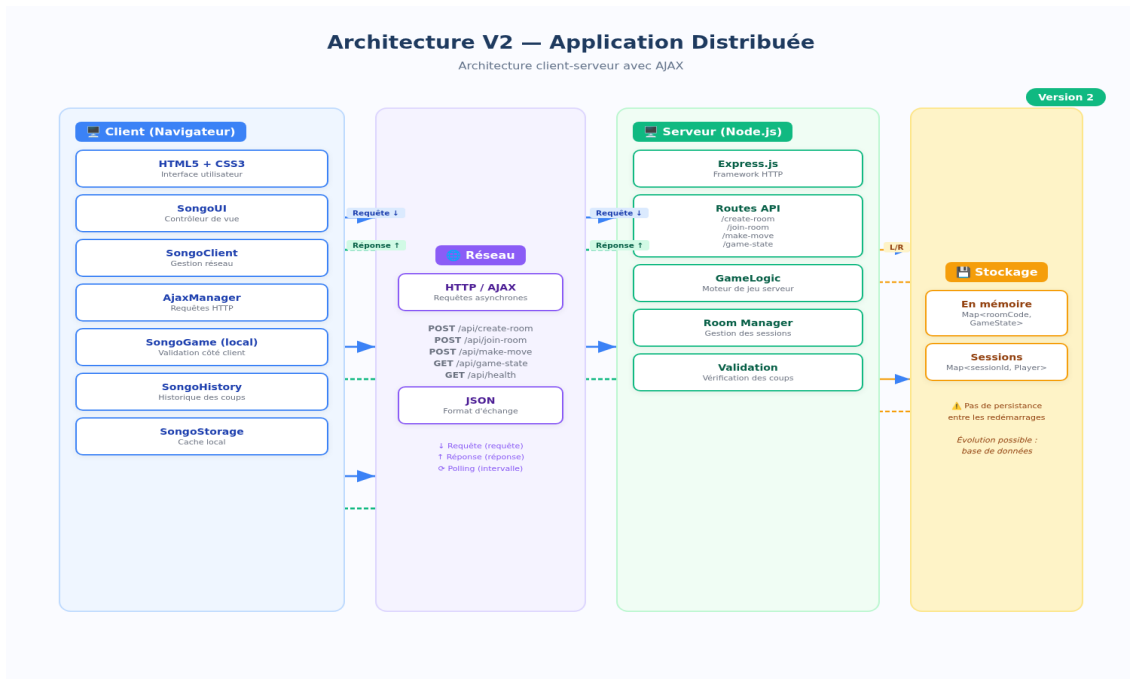


Figure 7 : Architecture client-serveur de la version distribuée

5.8 Modele de donnees

Le modele de donnees definit les structures utilisees pour représenter l'etat du jeu dans les deux versions. GameState contient l'etat complet du jeu (plateau, joueur courant, scores, historique, statut). MoveRecord enregistre chaque coup joue. RoomState (V2) représente un salon de jeu avec ses deux joueurs. SessionData (V2) associe une session a un salon et un numero de joueur. Ces structures sont echangees entre les composants sous forme JSON.

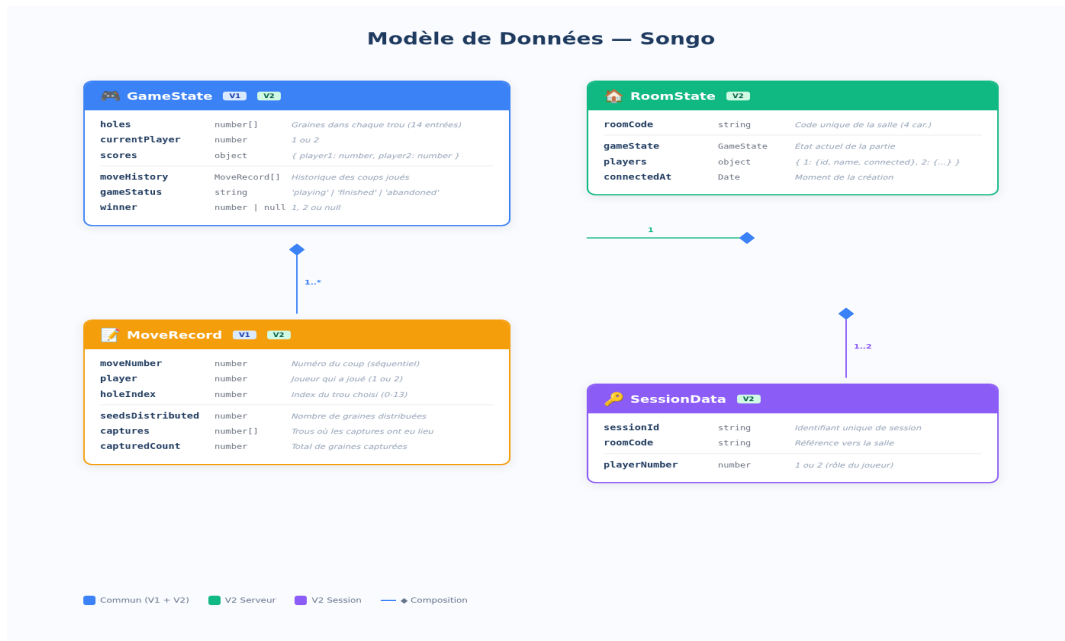


Figure 8 : Modèle de données du système Songo

6. Guide d'installation et d'utilisation

6.1 Version locale

Prerequis

La version locale ne nécessite aucune installation particulière. Il suffit d'un navigateur web moderne (Chrome, Firefox, Safari, Edge) supportant HTML5, CSS3 et JavaScript. Aucune dépendance externe ni serveur n'est requis.

Installation

1. Ouvrir le fichier `version_locale/html/index.html` dans un navigateur
2. La partie se lance automatiquement avec le plateau initialisé

Utilisation

- **Jouer un coup** : Cliquer sur un trou de votre rangée (non vide et surligné)
- **Nouvelle partie** : Cliquer sur le bouton 'Nouvelle Partie'
- **Sauvegarder** : Cliquer sur 'Sauvegarder' (stockage dans le navigateur)
- **Charger** : Cliquer sur 'Charger' pour reprendre une partie sauvegardée
- **Thème** : Cliquer sur l'icône soleil/lune pour basculer clair/sombre
- **Aide** : Cliquer sur l'icône d'aide pour afficher les règles

6.2 Version distribuée

Prerequis

La version distribuee necessite Node.js (version 14 ou superieure) et npm installes sur la machine qui hebergera le serveur. Les deux joueurs ont besoin d'un navigateur web moderne et d'un acces reseau au serveur.

Installation

```
1. cd version_distante/serveur/
2. npm install
3. node server.js
4. Le serveur demarre sur le port 3000
5. Ouvrir http://localhost:3000 dans deux navigateurs differents
```

Utilisation

- **Creer un salon** : Un joueur clique sur 'Creer une partie' et recoit un code
- **Rejoindre** : L'autre joueur entre le code et clique sur 'Rejoindre'
- **Jouer** : Les tours alternent, le plateau se met a jour automatiquement
- **Deconnexion** : Un message s'affiche si l'adversaire se deconnecte

API du serveur

Methode	Route	Description	Parametres
POST	/api/create-room	Creer un salon de jeu	-
POST	/api/join-room	Rejoindre un salon	roomCode
GET	/api/game-state	Obtenir l'etat du jeu	roomCode
POST	/api/make-move	Jouer un coup	roomCode, playerNumber, holeIndex
GET	/api/check-connection	Verifier la connexion	roomCode
POST	/api/leave-room	Quitter le salon	roomCode, playerNumber

Tableau 4 : API REST du serveur de la version distribuee

7. Annexes techniques

7.1 Table de distribution des graines

Le tableau suivant montre l'ordre de distribution pour chaque joueur. Cet ordre determine le parcours antihoraire des graines sur le plateau.

Position	Joueur 1 (ordre)	Joueur 2 (ordre)
1	Trou 0 (depart)	Trou 13 (depart)
2	Trou 1	Trou 12
3	Trou 2	Trou 11
4	Trou 3	Trou 10
5	Trou 4	Trou 9
6	Trou 5	Trou 8
7	Trou 6	Trou 7
8	Trou 13	Trou 6
9	Trou 12	Trou 5
10	Trou 11	Trou 4
11	Trou 10	Trou 3
12	Trou 9	Trou 2
13	Trou 8	Trou 1
14	Trou 7	Trou 0

Tableau 5 : Ordre de distribution antihoraire

7.2 Table des conditions de capture

Condition	Resultat	Exemple
Derniere graine dans camp adverse	Verification de capture	J1 distribue, derniere graine en trou 9
Trou contient 2 graines apres distribution	Capture de 2 graines	Trou 9 avait 1 graine, $+1 = 2$
Trou contient 3 graines apres distribution	Capture de 3 graines	Trou 9 avait 2 graines, $+1 = 3$
Trou contient 4+ graines apres distribution	Pas de capture	Trou 9 avait 3 graines, $+1 = 4$
Trou precedent a 2 ou 3 graines (meme camp)	Capture en chaine	Trou 10 a 2 graines, capture aussi
Capture laisserait adversaire sans graines	Capture arretee avant	On saute le dernier trou

Tableau 6 : Conditions de capture detaillees

7.3 Formats de donnees echanges

Format GameState (JSON)

```
{
  "holes": [5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5],
  "currentPlayer": 1,
  "scores": { "1": 0, "2": 0 },
  "moveHistory": [],
  "gameStatus": "playing",
  "winner": null,
  "moveCount": 0
}
```

Format MoveRecord (JSON)

```
{
  "moveNumber": 1,
  "player": 1,
  "holeIndex": 3,
  "seedsDistributed": 5,
  "captures": [10, 11],
  "capturedCount": 4,
  "lastHole": 11
}
```